

Angular Hands-On Exercises

```
// course.model.ts
export interface
  Course { id: number;
  name: string;
  code: string;
  credits:
  number;
  gradeStatus: 'passed' | 'failed' | 'pending';
}
```

```
// course.service.ts
import { Injectable } from
 '@angular/core'; import { Course }
from '../models/course.model';
```

```
@Injectable({
  providedIn:
  'root'
})
export class CourseService {
```

```
  private courses: Course[] = [
    {
      id: 1,
      name: 'Data
      Structures', code:
      'CS101',
      credits: 4,
      gradeStatus:
      'passed'
    },
    {
      id: 2,
      name: 'Database
      Management', code:
      'CS102',
      credits: 4,
      gradeStatus:
      'passed'
    },
    {
      id: 3,
      name: 'Computer
      Networks', code:
      'CS103',
      credits: 3,
      gradeStatus:
      'pending'
    },
    {
      id: 4,
      name: 'Operating
      Systems', code:
      'CS104',
```

```

        credits: 4,
        gradeStatus:
        'failed'
    },
    {
        id: 5,
        name: 'Web
        Development',
        code: 'CS105',
        credits: 3,
        gradeStatus:
        'passed'
    }
];

```

```

getCourses():
    Course[] { return
    this.courses;
}

```

```

getCourseById(id: number): Course | undefined {
    return this.courses.find(course => course.id === id);
}

```

```

addCourse(course: Course): void {
    this.courses.push(course);
}
}

```

```

// course-list.component.ts
import { Component, OnInit } from
'@angular/core'; import { Course } from
'../models/course.model';
import { CourseService } from '../services/course.service';

```

```

@Component({
    selector: 'app-course-list',
    templateUrl: '../course-list.component.html'
})

```

```

export class CourseListComponent

```

```

    implements OnInit { courses: Course[] =

```

```

    [];

```

```

    constructor(
        private courseService: CourseService
    ) {}

```

```

    ngOnInit(): void {
        this.courses = this.courseService.getCourses();
    }
}

```

```
// home.component.ts
import { Component, OnInit } from '@angular/core';
import { CourseService } from '../services/course.service';
```

```
@Component({
  selector: 'app-home',
  templateUrl: './home.component.html'
})
export class HomeComponent
```

```
  implements OnInit { courseCount = 0;
```

```
  constructor(
    private courseService: CourseService
  ) {}
```

```
  ngOnInit(): void {
    this.courseCount =
      this.courseService.getCourses().length;
  }
}
```

```
//
course-summary-widget.component.ts
import { Component } from
 '@angular/core';
import { CourseService } from '../services/course.service';
```

```
@Component({
  selector:
    'app-course-summary-widget',
  template: `
    <p>Total Courses: {{ courseService.getCourses().length }}</p>
  `
})
```

```
export class CourseSummaryWidgetComponent {
```

```
  constructor(
    public courseService: CourseService
  ) {}
}
```

```
// enrollment.service.ts
import { Injectable } from
 '@angular/core'; import { Course } from
 '../models/course.model'; import {
CourseService } from './course.service';
```

```
@Injectable({
  providedIn:
    'root'
})
export class EnrollmentService {
  private enrolledCourseIds: number[] =
    []; constructor(
    private courseService: CourseService
  ) {}
}
```

```

enroll(courseId: number): void {
  if
    (!this.enrolledCourseIds.includes(courseId)) {
      this.enrolledCourseIds.push(courseId);
    }
}

unenroll(courseId: number): void {
  this.enrolledCourseIds =
    this.enrolledCourseIds.filter(id => id !== courseId);
}

isEnrolled(courseId: number): boolean {
  return this.enrolledCourseIds.includes(courseId);
}

getEnrolledCourses():
  Course[] { return
    this.enrolledCourseIds
      .map(id => this.courseService.getCourseById(id))
      .filter((course): course is Course => course !== undefined);
  }
}

// course-card.component.ts
import { Component, Input } from
  '@angular/core'; import { Course } from
  '../models/course.model';
import { EnrollmentService } from '../services/enrollment.service';

@Component({
  selector:
    'app-course-card',
  template: `
    <h3>{{ course.name }}</h3>

    <button (click)="toggleEnrollment()">
      {{ enrollmentService.isEnrolled(course.id)
        ? 'Unenroll'
        : 'Enroll' }}
    </button>
  `
})
export class CourseCardComponent {

```

```

@Input() course!: Course;

constructor(
  public enrollmentService: EnrollmentService
) {}

toggleEnrollment(): void {
  if (this.enrollmentService.isEnrolled(this.course.id)) {
    this.enrollmentService.unenroll(this.course.id);
  } else {
    this.enrollmentService.enroll(this.cours
    e.id);
  }
}
}

// student-profile.component.ts
import { Component } from '@angular/core';
import { EnrollmentService } from '../services/enrollment.service';

@Component({
  selector:
    'app-student-profile',
  template: `
    <h2>Enrolled Courses</h2>

    <div *ngFor="let course of enrollmentService.getEnrolledCourses()">
      {{ course.name }}
    </div>
  `
})
export class StudentProfileComponent {

  constructor(
    public enrollmentService: EnrollmentService
  ) {}
}

// notification.service.ts
import { Injectable } from '@angular/core';

@Injectable()
export class NotificationService {

  show(message: string): void {
    console.log(message);
  }
}

// notification.component.ts
import { Component } from '@angular/core';
import { NotificationService } from '../services/notification.service';

@Component({
  selector: 'app-notification',
  template: `<p>Notification Component</p>`,

```

```
// Component-level provider creates a new
// NotificationService instance scoped to this
// component and its child
components. providers:
  [NotificationService]
})
export class NotificationComponent {

  constructor(
    private notificationService: NotificationService
  ) {}
}
```

```
// app.routes.ts
import { Routes } from '@angular/router';
import { HomeComponent } from './pages/home.component';
import { CourseListComponent } from
'./pages/course-list.component'; import {
CourseDetailComponent } from
'./pages/course-detail.component';
import { StudentProfileComponent } from
'./pages/student-profile.component'; import {
NotFoundComponent } from './pages/not-found.component';
import { CoursesLayoutComponent } from
'./pages/courses-layout.component'; import { AuthGuard }
from './guards/auth.guard';
import { UnsavedChangesGuard } from

'./guards/unsaved-changes.guard'; export const routes:
```

```
Routes = [

  {
    path: "",
    component: HomeComponent
  },

  {
    path: 'courses',
    component:
CoursesLayoutComponent,
    children: [
      {
        path: "",
        component: CourseListComponent
      },
      {
        path: ':id',
        component: CourseDetailComponent
      }
    ]
  },

  {
    path: 'profile',
    canActivate:
[AuthGuard],
    component: StudentProfileComponent
  },

  {
    path: 'enroll',
    canActivate:
[AuthGuard],
    canDeactivate:
[UnsavedChangesGuard],
    loadChildren: () =>
import('./features/enrollment/enrollment.module')
.then(m => m.EnrollmentModule)
  },

```

```

    {
      path: '**',
      component: NotFoundComponent
    }
  ];

// course-detail.component.ts
import { Component, OnInit } from
'@angular/core'; import { ActivatedRoute }
from '@angular/router'; import { Course }
from '../models/course.model';
import { CourseService } from '../services/course.service';

@Component({
  selector:
'app-course-detail',
  template: `
    <div *ngIf="course">
      <h2>{{ course.name }}</h2>
      <p>Code: {{ course.code }}</p>
      <p>Credits: {{ course.credits }}</p>
      <p>Status: {{ course.gradeStatus }}</p>
    </div>
  `
})
export class CourseDetailComponent

  implements OnInit { course?: Course;

  constructor(
    private route: ActivatedRoute,
    private courseService:
      CourseService
  ) {}

  ngOnInit(): void
  { const id =
    Number(
      this.route.snapshot.paramMap.get('id')
    );

    this.course =
      this.courseService.getCourseById(
        id);
  }
}

// course-list.component.ts
import { Router, ActivatedRoute } from

'@angular/router'; searchTerm = "";

```



```

constructor(
  private router: Router,
  private route:
    ActivatedRoute,
  private courseService: CourseService
) {}

```

```

openCourse(courseId: number): void {
  this.router.navigate([
    'courses'
    ,
    courseId
  ]);
}

```

```

searchCourses(): void
{
  this.router.navigate(
    ['courses'],
    {
      queryParams: {
        search: this.searchTerm
      }
    }
  );
}

```

```

readSearch(): void
{
  const search =
    this.route.snapshot.queryParamMap
      .get('search');

  console.log(search);
}

```

```

// courses-layout.component.html
<router-outlet></router-outlet>

```

```

// auth.service.ts
import { Injectable } from '@angular/core';

```

```

@Injectable({
  providedIn:
    'root'
})
export class AuthService {

```

```

  isLoggedIn = true;
}

```

```

// auth.guard.ts
import { inject } from '@angular/core';
import { CanActivateFn, Router } from
  '@angular/router'; import { AuthService } from
  '../services/auth.service';

```

```

export const AuthGuard: CanActivateFn = () => {

```

```

const authService =
inject(AuthService); const router
= inject(Router);

if
  (authService.isLoggedIn
  In) { return true;
}

router.navigate(['/']);
return false;
};

// unsaved-changes.guard.ts
import { CanDeactivateFn } from '@angular/router';

export interface
  CanComponentDeactivate {
    enrollForm: {
      dirty: boolean;
    };
  }

export const UnsavedChangesGuard:
  CanDeactivateFn<CanComponentDea
  ctivate> = (component) => {

  if
    (component.enrollForm.dirty)
    { return window.confirm(
      'You have unsaved changes. Leave?'
    );
    }

  return true;
};

```

```

// course.service.ts - HttpClient
Version import { Injectable } from
'@angular/core';
import { HttpClient } from
'@angular/common/http'; import {
Observable } from 'rxjs';
import { map, catchError, tap, retry } from
'rxjs/operators'; import { throwError } from 'rxjs';
import { Course } from '../models/course.model';

@Injectable({
  providedIn:
  'root'
})
export class CourseService {

  private apiUrl =
    'http://localhost:3000/courses';

  constructor(
    private http: HttpClient
  ) {}

  getCourses():
  Observable<Course[]> { return
  this.http
    .get<Course[]>(this.apiUrl)
    .pipe(
      map(courses =>
        courses.filter(cours
          e =>
            course.credits > 0
        )
      ),
      tap(courses
        =>
          console.lo
            g(
              'Courses
                loaded:',
                courses.length
            )
          ),
      retry(2),
      catchError(err
        => {
          console.error(err);

          return
            throwError(() =>
              new Error(
                'Failed to load courses. Please try again.'
              )
            );
        })
    );
  }
}

```

```

getCourseById( id:
  number
): Observable<Course> {

  return this.http.get<Course>(
    `${this.apiUrl}/${id}`
  );
}

```

```

createCourse(
  course: Omit<Course, 'id'>
): Observable<Course> {

  return
    this.http.post<Course>
      ( this.apiUrl,
        course
      );
}

```

```

updateCourse( id:
  number,
  course: Partial<Course>
): Observable<Course> {

  return this.http.put<Course>(
    `${this.apiUrl}/${id}`, course
  );
}

```

```

deleteCourse( id:
  number
): Observable<void> {

  return this.http.delete<void>(
    `${this.apiUrl}/${id}`
  );
}

```

```

// course-list.component.ts - HTTP Version
import { Component, OnInit } from
'@angular/core'; import { Course } from
'../models/course.model';
import { CourseService } from

'../services/course.service'; export class

```

CourseListComponent

```

implements OnInit {

  courses: Course[] =
  []; errorMessage = "";
  isLoading = true;

  constructor(
    private courseService: CourseService
  ) {}
  ngOnInit(): void {
    this.courseService
      .getCourses()
      .subscribe({
        next: courses =>
          this.courses =
            courses,

        error: err => {
          this.errorMessage =
            err.message; this.isLoading =
            false;
        },

        complete: () =>
          this.isLoading =
            false
      });
  }
}

// enrollment.service.ts - HTTP
Version import { Injectable } from
'@angular/core';
import { HttpClient } from
'@angular/common/http'; import {
Observable } from 'rxjs';
import { Course } from '../models/course.model';

@Injectable({
  providedIn:
  'root'
})
export class EnrollmentService {

  constructor(
    private http: HttpClient
  ) {}

  getStudentsByCourse(
    courseId:
    number
  ): Observable<any[]> {

    return this.http.get<any[]>(`
    http://localhost:3000/enrollments?courseId=${courseId}`
    );
  }
}

```

```

}

// course-student.service.ts
import { Injectable } from
 '@angular/core'; import { switchMap }
from 'rxjs/operators'; import {
CourseService } from './course.service';
import { EnrollmentService } from './enrollment.service';

@Injectable({
  providedIn:
    'root'
})
export class CourseStudentService {

  constructor(
    private courseService:
      CourseService, private
      enrollmentService:
      EnrollmentService
  ) {}
  getStudentsForCourse(courseId:
    number) { return
    this.courseService
      .getCourseById(courseId)
      .pipe(
        switchMap(course =>
          this.enrollmentService
            .getStudentsByCourse(course.id)
        )

        // switchMap cancels the previous inner
        // Observable when a new courseId arrives.
      );
    }
  }
}

// auth.interceptor.ts
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor:
  HttpInterceptorFn = (req, next)
=> {

  const authReq =
    req.clone({
      setHeaders: {
        Authorization:

```

```

        'Bearer mock-token-12345'
    }
  });

  return next(authReq);
};

// error-handler.interceptor.ts
import { HttpInterceptorFn } from
'@angular/common/http'; import { catchError,
throwError } from 'rxjs';
import { Router } from
'@angular/router'; import { inject }
from '@angular/core';

export const errorHandlerInterceptor:
HttpInterceptorFn = (req, next) => {
  const router =
    inject(Router); return
    next(req).pipe(
      catchError(error => {

        if (error.status === 401) {
          router.navigate(['/']);
        }

        if (error.status ===
          500) { console.error(
            'Global server error'
          );
        }

        return throwError(() => error);
      })
    );
};

// loading.service.ts
import { Injectable } from
'@angular/core'; import {
BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn:
    'root'
})
export class LoadingService {

  private loadingSubject =
    new BehaviorSubject<boolean>(false);

  isLoading$ =
    this.loadingSubject.asObservable();

  setLoading(value: boolean): void {
    this.loadingSubject.next(value);
  }
}

```

```

}

// loading.interceptor.ts
import { HttpInterceptorFn } from
'@angular/common/http'; import { finalize } from
'rxjs';
import { inject } from '@angular/core';
import { LoadingService } from '../services/loading.service';

export const loadingInterceptor:
  HttpInterceptorFn = (req, next) => {

  const loadingService
    =
    inject(LoadingServ
      ice);
  loadingService.setLoading
    (true); return
    next(req).pipe(
      finalize(() => {
        loadingService.setLoading(false);
      })
    );
};

// app.config.ts
import {
  provideHttpClient,
  withInterceptors
} from '@angular/common/http';

import {
  authIntercep
    tor
  } from './interceptors/auth.interceptor';

import {
  errorHandlerIntercep
    tor
  } from './interceptors/error-handler.interceptor';

import {
  loadingIntercept
    or
  } from './interceptors/loading.interceptor';

```



```
export const
  appConfig = {
    providers: [
      provideHttpClient(
        withInterceptors(
          [
            authInterceptor,
            errorHandlerIntercept
            or, loadingInterceptor
          ]
        )
      )
    ]
  };
```

```
// package installation
npm install @ngrx/store @ngrx/effects @ngrx/entity @ngrx/store-devtools
```

```
// course.actions.ts
import { createAction, props } from
'@ngrx/store'; import { Course } from
'../../models/course.model';
```

```
export const loadCourses =
  createAction('[Course] Load
  Courses');
```

```
export const
  loadCoursesSuccess =
  createAction(
    '[Course] Load Courses
    Success', props<{ courses:
    Course[] }>()
  );
```

```
export const
  loadCoursesFailure =
  createAction(
    '[Course] Load Courses
    Failure', props<{ error: string
    }>()
  );
```

```
// course.reducer.ts
import { createReducer, on } from
'@ngrx/store'; import {
  loadCourses,
  loadCoursesSuccess,
  loadCoursesFailure
} from './course.actions';
import { Course } from '../../models/course.model';
```

```
export interface
  CourseState { courses:
  Course[];
  loading: boolean;
  error: string | null;
}
```

```
export const initialState: CourseState =
  { courses: [],
  loading: false,
  error: null
};
```

```
export const
  courseReducer =
  createReducer(
    initialState,

    on(loadCourses, state => ({
      ...state,
```

```

        loading:
        true, error:
        null
    })),

    on(
        loadCoursesSuccess,
        (state, { courses })
        => ({
            ...state,
            courses,
            loading:
            false
        })
    ),

    on(
        loadCoursesFailure
        , (state, { error })
        => ({
            ...state,
            loading:
            false, error
        })
    )
);

// course.selectors.ts
import {
    createFeatureSelecto
    r, createSelector
} from '@ngrx/store';
import { CourseState } from
'./course.reducer'; export const
selectCourseState =
    createFeatureSelector<CourseState>('course');

export const
selectAllCourses =
createSelector(
    selectCourseState,
    state =>
    state.courses
);

export const selectCoursesLoading =
createSelector(
    selectCourseState,
    state =>
    state.loading
);

```

```

export const
  selectCoursesError =
  createSelector(
    selectCourseState,
    state =>
      state.error
  );

// course.effects.ts
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from
'@ngrx/effects'; import {
  catchError
  or, map,
  of,
  switch
  Map
} from 'rxjs';
import { CourseService } from
'../../services/course.service'; import {
  loadCourses,
  loadCoursesSuccess,
  loadCoursesFailure
} from './course.actions';

```

```

@Injectable()
export class CourseEffects {

  loadCourses$ = createEffect(() =>
    this.actions$.pipe(
      ofType(loadCourses),

      switchMap(() =>
        this.courseService
          .getCourses()
          .pipe(

            map(courses =>
              loadCoursesSuccess({
                courses })
            ),

            catchError(error
              => of(
                loadCoursesFailure(
                  { error:
                    error.message
                })
              )
            )
          )
        )
      )
    );

```

```

constructor(
  private actions$: Actions,

```

```
    private courseService: CourseService
  ) {}
}
```

```
// enrollment.actions.ts
import { createAction, props } from '@ngrx/store';
```

```
export const
  enrollInCourse =
    createAction(
      '[Enrollment] Enroll In Course',
      props<{ courseId: number }>()
    );
```

```
export const
  unenrollFromCourse =
    createAction(
      '[Enrollment] Unenroll From
      Course', props<{ courseId:
      number }>()
    );
```

```
export const
  setEnrolledCourses =
    createAction(
      '[Enrollment] Set Enrolled
      Courses', props<{ courseIds:
      number[] }>()
    );
```

```
// enrollment.reducer.ts
import { createReducer, on } from
 '@ngrx/store'; import {
  enrollInCourse,
  unenrollFromCourse
  , setEnrolledCourses
} from './enrollment.actions';
```

```
export interface EnrollmentState {
  enrolledCourseIds: number[];
}
```

```
export const initialEnrollmentState:
  EnrollmentState = {
    enrolledCourseIds: []
  };
};
```

```

export const
  enrollmentReducer =
  createReducer(
    initialEnrollmentState,

    on(
      enrollInCourse,
      (state, { courseId }) => ({
        ...state,
        enrolledCourse
        Ids:
          state.enrolledCourseIds.includes(courseId)
            ? state.enrolledCourseIds
            : [
              ...state.enrolledCourseIds,
              courseId
            ]
      })
    ),

    on(
      unenrollFromCourse,
      (state, { courseId }) => ({
        ...state,
        enrolledCourse
        Ids:
          state.enrolledCourseIds.fil
            ter( id => id !==
              courseId
            )
      })
    ),

    on(
      setEnrolledCourses,
      (state, { courseIds }) => ({
        ...state,
        enrolledCourseIds: courseIds
      })
    )
  );

```

```

// enrollment.selectors.ts
import { createFeatureSelector } from
  '@ngrx/store'; import { EnrollmentState } from
  './enrollment.reducer';

```

```

export const selectEnrollmentState =
  createFeatureSelector<EnrollmentStat
  e>(
    'enrollment'
  );

```

```

export const
  selectEnrolledIds = (state:
  EnrollmentState) =>
  state.enrolledCourseIds;

```

```

// course-card.component.ts - NgRx
import { Component, Input } from
 '@angular/core'; import { Store } from
 '@ngrx/store';
import {
  enrollInCourse,
  unenrollFromCo
  urse
} from '../store/enrollment/enrollment.actions';

import {
  selectEnrolled
  Ids
} from

 '../store/enrollment/enrollment.selectors';

import { Course } from

 '../models/course.model';

@Component({
  selector:
    'app-course-card',
  template: `
    <h3>{{ course.name }}</h3>

    <button
      (click)="toggleEnrollment(
    )">

      {{
        (enrolledIds$ | async)?.includes(course.id)
          ? 'Unenroll'
          : 'Enroll'
      }}

    </button>
  `,
})
export class

  CourseCardComponent {

    @Input() course!: Course;

    enrolledIds$ =
      this.store.select(selectEnrolledIds);

    constructor(
      private store: Store
    ) {}

    toggleEnrollment(): void {

      this.store.dispatch(

```

```

        enrollInCourse({
            courseId: this.course.id
        })
    );
}
}

//
course-card.component.spec
.ts import {
    ComponentFix
    ture, TestBed
} from '@angular/core/testing';
import { By } from
'@angular/platform-browser'; import {
    CourseCardComponent
} from

'./course-card.component';

describe('CourseCardComponen

t', () => {

    let component:
    CourseCardComponent; let
    fixture:
    ComponentFixture<CourseCardComponent>;

    const
    mockCourse =
    { id: 1,
    name: 'Data
    Structures', code:
    'CS101',
    credits: 4,
    gradeStatus: 'passed' as const
    };

    beforeEach(async () => {

        await
        TestBed.configureTestingModule(
            { imports: [
                CourseCardComponent
            ]
        }).compileComponents();

        fixture =
        TestBed.createCom
        ponent(
            CourseCardComponent
        );

        component =
        fixture.componentInsta
        nce;
    });

```



```

it('should create', () => {
  expect(component).toBeTruthy();
});

it('should display course name', () =>

  { component.course = mockCourse;

  fixture.detectChanges();

  const title =
    fixture.debugEle
    ment
    .query(By.css('h3'))
    .nativeElement;

  expect(title.textContent)
    .toContain('Data Structures');
});

it('should emit enroll event', () => {

  component.course = mockCourse;

  fixture.detectChanges();

  spyOn(
    component.enrollRequ
    ested, 'emit'
  );

  fixture.debugElement
    .query(By.css('button'))
    .nativeElement
    .click();

  expect(
    component.enrollRequeste
    d.emit
  ).toHaveBeenCalledWith(1);
});

//
course.service.spec.ts
import {
  TestBed
} from '@angular/core/testing';

import {
  HttpClientTestingModule,
  HttpTestingControll
  er
} from '@angular/common/http/testing';

import { CourseService } from

```

```
 './course.service'; describe('CourseService',
```

```
 () => {
```

```
   let service: CourseService;
```

```
   let httpMock: HttpTestingController;
```

```
   const mockCourses = [
```

```
     {
```

```
       id: 1,
```

```
       name: 'Data
```

```
       Structures', code:
```

```
       'CS101',
```

```
       credits: 4,
```

```
       gradeStatus: 'passed' as const
```

```
     },
```

```
     {
```

```
       id: 2,
```

```
       name: 'Database',
```

```
       code: 'CS102',
```

```
       credits: 4,
```

```
       gradeStatus: 'passed' as const
```

```
     }  
   ];
```

```
   beforeEach(() => {
```

```
     TestBed.configureTestingModule({
```

```
       imports: [
```

```
         HttpClientTestingModule
```

```
       ],
```

```
       providers: [
```

```
         CourseSer
```

```
         vice
```

```
       ]
```

```
     });
```

```
     service =
```

```
       TestBed.inject(CourseService
```

```
     );
```

```
     httpMock =
```

```
       TestBed.inj
```

```
       ect(
```

```
         HttpTestingController
```

```
       );
```

```
   });
```

```

it('should get courses', () => {

  service.getCourses()
    .subscribe(courses => {
      expect(courses.length)
        .toBe(2);
    });

  const request =
    httpMock.expect
      One(
        'http://localhost:3000/courses'
      );

  expect(request.request.method)
    .toBe('GET');
  request.flush(mockCourses);
  httpMock.verify();
});

it('should handle server error', () => {
  service.getCourses()
    .subscribe({
      next: () => fail('Expected error'),
      error: error => {
        expect(error.message)
          .toBe(
            'Failed to load courses. Please try again.'
          );
      }
    });

  const request =
    httpMock.expect
      One(
        'http://localhost:3000/courses'
      );

  request.flush(
    'Server
    Error',
    {
      status: 500,
      statusText: 'Server Error'
    }
  );

  httpMock.verify();
});
});

// course-list.component.spec.ts -
MockStore import {
  TestBed
} from '@angular/core/testing';

import {
  provideMockS
  tore,

```

```

    MockStore
  } from '@ngrx/store/testing';

import {
  CourseListComponent
} from './course-list.component';

describe('CourseListComponent', () => {

  let store: MockStore;

  const initialState = {
    course: {
      courses: [
        {
          id: 1,
          name: 'Data Structures', code:
            'CS101',
          credits: 4,
          gradeStatus:
            'passed'
        }
      ],
      loading:
        false, error:
        null
    }
  };
  beforeEach(() => {
    TestBed.configureTestingModule({
      imports: [
        CourseListComponent
      ],
      providers: [
        provideMockStore(
          {
            initialState

```

```

    })
  ]
});

store =
  TestBed.inject(MockStore);
});
it('should show loading state', () => {
  store.setState({
    course: {
      courses: [],
      loading:
        true, error:
        null
    }
  });

  expect(
    TestBed.inject(Store)
  ).toBeTruthy();
});
});

```